

Prompt: Cargos Predefinidos con Cantidad en el Folio

Contexto

En el folio de una reserva (tab "Folio" dentro de "Ver completo"), al agregar cargos hoy se escribe descripción + monto manualmente. Se necesita:

1. **Lista de cargos predefinidos** (cochera, media pensión, etc.) para seleccionar en lugar de escribir
2. **Campo de cantidad** que multiplica automáticamente el precio unitario
3. **Editar cantidad** de un cargo ya cargado directamente en la lista, sin abrir un modal

PASO 1 — Definir los cargos predefinidos

En `ReservationFormDialog`, justo antes o después de los estados existentes del folio, agregar la lista de presets:

```
const chargePresets = [  
  { label: "Cochera (por día)",      description: "Cochera",          amount: "25",  
    { label: "Media Pensión",        description: "Media Pensión",      amount: "4",  
    { label: "Pensión Completa",      description: "Pensión Completa",  amount: "8",  
    { label: "Desayuno adicional",     description: "Desayuno adicional", amount: "1",  
    { label: "Cena",                  description: "Cena",              amount: "3",  
    { label: "Frigobar",              description: "Frigobar",          amount: "1",  
    { label: "Lavandería",            description: "Lavandería",        amount: "2",  
    { label: "Traslado",              description: "Traslado",          amount: "3",  
    { label: "SPA / Masaje",           description: "SPA / Masaje",      amount: "5",  
    { label: "Cargo manual",          description: "",                  amount: ""  
];
```

Nota: los montos son ejemplos, el recepcionista puede editarlos. "Cargo manual" es la opción libre (comportamiento actual).

Agregar también el estado para la cantidad:

```
const [chargeQty, setChargeQty] = useState(1);
const [selectedPreset, setSelectedPreset] = useState<typeof chargePresets[0] | nu
```

PASO 2 — Reemplazar el formulario de agregar cargo

Buscar el bloque `{showAddCharge && (` dentro del tab Folio y reemplazarlo por este nuevo formulario:

```
{showAddCharge && (
  <div className="p-3 border-b bg-muted/30 space-y-3">

    {/* Fila 1: selector de preset */}
    <div>
      <Label className="text-xs text-muted-foreground mb-1 block">Tipo de cargo</
      <Select
        value={selectedPreset?.label || ""}
        onChange={(val) => {
          const preset = chargePresets.find(p => p.label === val);
          if (preset) {
            setSelectedPreset(preset);
            setNewCharge({
              description: preset.description,
              amount: preset.amount,
              category: preset.category,
            });
            setChargeQty(1);
          }
        }}
      >
        <SelectTrigger data-testid="select-charge-preset">
          <SelectValue placeholder="Seleccionar tipo..." />
        </SelectTrigger>
        <SelectContent>
          {chargePresets.map((p) => (
            <SelectItem key={p.label} value={p.label}>
              {p.label}
            </SelectItem>
          ))}
        </SelectContent>
      </Select>
    </div>

    {/* Fila 2: descripción, precio unitario, cantidad, total */}
```

```

<div className="grid grid-cols-4 gap-2 items-end">
  <div className="col-span-2">
    <Label className="text-xs text-muted-foreground mb-1 block">Descripción</Label>
    <Input
      placeholder="Descripción del cargo"
      value={newCharge.description}
      onChange={(e) => setNewCharge({ ...newCharge, description: e.target.value })}
      data-testid="input-charge-description"
    />
  </div>

  <div>
    <Label className="text-xs text-muted-foreground mb-1 block">Precio unit.</Label>
    <Input
      type="number"
      placeholder="0.00"
      value={newCharge.amount}
      onChange={(e) => {
        setNewCharge({ ...newCharge, amount: e.target.value });
      }}
      data-testid="input-charge-unit-price"
    />
  </div>

  <div>
    <Label className="text-xs text-muted-foreground mb-1 block">Cantidad</Label>
    <Input
      type="number"
      min="1"
      step="1"
      value={chargeQty}
      onChange={(e) => setChargeQty(Math.max(1, parseInt(e.target.value) || 1))}
      data-testid="input-charge-qty"
    />
  </div>
</div>

{ /* Total calculado */ }
{ newCharge.amount && chargeQty > 1 && (
  <div className="text-sm text-right text-muted-foreground">
    Total: <span className="font-semibold text-foreground">
      ${parseFloat(newCharge.amount || "0") * chargeQty}.toFixed(2)}
    </span>
    <span className="ml-1 text-xs">
      ({chargeQty} × ${parseFloat(newCharge.amount || "0").toFixed(2)})
    </span>
  </div>
)}

```

```

    })

    { /* Categoría */ }
    <div className="grid grid-cols-2 gap-2">
      <div>
        <Label className="text-xs text-muted-foreground mb-1 block">Categoría</La
        <Select
          value={newCharge.category}
          onChange={(value) => setNewCharge({ ...newCharge, category: value
        >
          <SelectTrigger data-testid="select-charge-category">
            <SelectValue />
          </SelectTrigger>
          <SelectContent>
            <SelectItem value="restaurant">Restaurante</SelectItem>
            <SelectItem value="spa">Spa</SelectItem>
            <SelectItem value="minibar">Minibar / Frigobar</SelectItem>
            <SelectItem value="otros">Otros</SelectItem>
            <SelectItem value="adjustment">Ajuste</SelectItem>
          </SelectContent>
        </Select>
      </div>
    </div>

    { /* Botones */ }
    <div className="flex justify-end gap-2">
      <Button
        size="sm"
        variant="ghost"
        onClick={() => {
          setShowAddCharge(false);
          setSelectedPreset(null);
          setChargeQty(1);
          setNewCharge({ description: "", amount: "", category: "otros" });
        }}
      >
        Cancelar
      </Button>
      <Button
        size="sm"
        onClick={() => {
          if (!newCharge.description || !newCharge.amount) return;
          const totalAmount = (parseFloat(newCharge.amount) * chargeQty).toFixed(
          const descWithQty = chargeQty > 1
            ? `${newCharge.description} (x${chargeQty})`
            : newCharge.description;
          addChargeMutation.mutate({

```

```

        description: descWithQty,
        amount: totalAmount,
        category: newCharge.category,
        reservationId: reservation.id,
        date: new Date().toLocaleDateString("en-CA", { timeZone: "America/Arg
    });
    setSelectedPreset(null);
    setChargeQty(1);
  }}
  disabled={!newCharge.description || !newCharge.amount || addChargeMutatio
  data-testid="button-confirm-charge"
>
  {addChargeMutation.isPending ? "Guardando..." : `Agregar${chargeQty > 1 ?
</Button>
</div>
</div>
)}

```

PASO 3 — Edición inline de cantidad en cargos ya cargados

En la lista `{consumptionCharges.map((charge) => (` reemplazar la fila de cada cargo para agregar edición inline de monto:

```

{consumptionCharges.map((charge) => (
  <div key={charge.id} className="flex items-center justify-between p-3 text-sm"
    <div className="flex items-center gap-2 flex-wrap flex-1 min-w-0">
      <Badge variant="outline" className="text-xs shrink-0">{categoryLabels[charg
      <span className="truncate">{charge.description}</span>
      <span className="text-muted-foreground text-xs shrink-0">({charge.date})</s
    </div>
    <div className="flex items-center gap-1 shrink-0 ml-2">
      {/* Edición inline del monto - click en el monto para editar */}
      {editingChargeId === charge.id ? (
        <div className="flex items-center gap-1">
          <span className="text-muted-foreground text-xs">${</span>
          <Input
            type="number"
            className="h-6 w-20 text-xs p-1"
            defaultValue={parseFloat(charge.amount).toFixed(2)}
            autoFocus
            onBlur={(e) => {
              const val = e.target.value;
              if (val && parseFloat(val) !== parseFloat(charge.amount)) {
                updateChargeMutation.mutate({ id: charge.id, amount: parseFloat(v

```

```

        }
        setEditingChargeId(null);
    }}
    onKeyDown={ (e) => {
        if (e.key === "Enter") (e.target as HTMLInputElement).blur();
        if (e.key === "Escape") setEditingChargeId(null);
    }}
    data-testid={`input-edit-charge-${charge.id}`}
  />
</div>
) : (
  <button
    className="font-medium hover:text-primary hover:underline cursor-pointe
    onClick={() => setEditingChargeId(charge.id)}
    title="Clic para editar monto"
    data-testid={`button-edit-charge-amount-${charge.id}`}
  >
    ${parseFloat(charge.amount).toFixed(2)}
  </button>
)}
{/* Botones transfer y delete (ya existentes) */}
<Button
  size="icon"
  variant="ghost"
  className="h-6 w-6"
  onClick={() => setTransferringChargeId(charge.id)}
  title="Transferir a otra habitación"
  data-testid={`button-transfer-charge-${charge.id}`}
>
  <ArrowRightLeft className="h-3 w-3 text-blue-600" />
</Button>
<Button
  size="icon"
  variant="ghost"
  className="h-6 w-6"
  onClick={() => deleteChargeMutation.mutate(charge.id)}
  data-testid={`button-delete-charge-${charge.id}`}
>
  <Trash2 className="h-3 w-3 text-destructive" />
</Button>
</div>
</div>
)})}

```

PASO 4 — Agregar estado y mutation para edición inline

Junto a los otros estados del folio, agregar:

```
const [editingChargeId, setEditingChargeId] = useState<string | null>(null);

const updateChargeMutation = useMutation({
  mutationFn: async ({ id, amount }: { id: string; amount: string }) => {
    return apiRequest("PATCH", `/api/charges/${id}`, { amount });
  },
  onSuccess: () => {
    refetchCharges();
    toast({ title: "Cargo actualizado" });
  },
  onError: () => {
    toast({ title: "Error al actualizar el cargo", variant: "destructive" });
  },
});
```

Resumen del comportamiento

- Al hacer clic en **“Agregar”** en el encabezado de Consumos, aparece el formulario expandido
- Se puede elegir un preset (cochera, media pensión, etc.) que pre-rellena descripción, precio y categoría
- El campo **Cantidad** multiplica el precio: si ponés 3 cocheras a \$2.500, se carga un solo ítem **“Cochera (x3)”** por \$7.500
- Si se elige **“Cargo manual”**, los campos quedan vacíos para completar libremente
- En la lista de cargos ya cargados, el **monto es clickable** → se convierte en un input inline → Enter o click afuera guarda → Escape cancela
- Los botones de transferir y eliminar siguen funcionando igual